Logistic Regression 2

การบรรยายที่ 11: การประเมิน classifier และ multinomial logistic regression

อาจารย์สิปปนนท์ กิติมูล

ภาคการศึกษาที่ 1/2569

model ที่ถูก 85% อาจพลาดผู้ป่วยเกือบทุกคน วันนี้เราจะวัดให้เห็น แก้ให้ตรงกับงาน แล้วขยายไปสู่มากกว่าสองกลุ่ม

เรามาถึงตรงนี้ได้อย่างไร

จบคาบที่ 10 ด้วย Framingham	
accuracy ของ model	0.8565
ทาย “ไม่เป็นโรค” ทุกคน	0.8477
ผู้ป่วยที่จับได้	51 จาก 557

คำถามที่ค้างไว้

1. วัดอย่างไรให้เห็นสิ่งที่ accuracy ซ่อนไว้
2. ตัวเลขนั้นวัดบน training set ที่ model เห็นมาแล้ว
3. ทำไมต้องตัดที่ 0.5
4. ถ้า Y มีมากกว่าสองกลุ่มล่ะ

model เดิมไม่ได้แย่ สัมประสิทธิ์ตีความได้ และ $\hat{p}$ เรียงลำดับความเสี่ยงได้ สิ่งที่ยังขาดคือ **วิธีวัด** และ **วิธีตัดสินใจ**

แผนที่ของคาบนี้

1. Confusion matrix: นับว่าถูกและผิดแบบไหน
2. Recall, precision, specificity และ F1
3. Train/test split: วัดบนข้อมูลที่ model ไม่เคยเห็น
4. Threshold: 0.5 ไม่ใช่กฎธรรมชาติ
5. ROC curve และ AUC: ดูทุก threshold พร้อมกัน
6. Multinomial logistic regression: เมื่อ Y มีสามกลุ่มขึ้นไป

อ่านประกอบ: ISLR §4.3.5 และ §4.4.3 ส่วนที่ว่าด้วย confusion matrix, threshold และ ROC (Table 4.4–4.7, Figure 4.7–4.8)

Confusion matrix

นับให้แยกว่าถูกแบบไหน และผิดแบบไหน

Confusion matrix บน `Default`

`default ~ balance` จากคาบที่ 10 ตัดที่ $\hat{p} > 0.5$ บนลูกค้าทั้ง 10000 คน

	ทายว่า “ไม่ผิดนัด” ($\hat{y} = 0$)	ทายว่า “ผิดนัด” ($\hat{y} = 1$)	รวม
จริง: ไม่ผิดนัด ($y = 0$)	TN $= 9625$	FP $= 42$	9667
จริง: ผิดนัด ($y = 1$)	FN $= 233$	TP $= 100$	333
รวม	9858	142	10000

- ตัวเลขชุดเดียวกับคาบที่ 10: ทาย “ผิดนัด” 142 คน ถูก 100 คน พลาดผู้ผิดนัดจริง 233 คน
- แค่จัดเป็นตาราง ก็เห็นว่า **ความผิดสองแบบ** มีจำนวนต่างกันมาก

สี่ช่อง สี่ชื่อ

อ่านชื่อจากสองคำ

- คำหลัง **Positive/Negative** คือสิ่งที่ model ทาย
- คำแรก **True/False** คือทายถูกหรือผิด
- “positive” คือกลุ่มที่เราสนใจ (ผิดนัด เป็นโรค) ไม่ได้แปลว่าดี

ความผิดสองแบบ

- **FP** (false alarm, Type I): ทายว่าผิดนัด แต่จริง ๆ ไม่ผิดนัด ธนาคารปฏิเสธลูกค้าดี
- **FN** (miss, Type II): ทายว่าไม่ผิดนัด แต่ผิดนัดจริง ธนาคารเสียเงิน

ความผิดแบบไหนแย่กว่า **ขึ้นกับงาน** ไม่ใช่ขึ้นกับสถิติ ตรวจคัดกรองโรค: FN แพงกว่ามาก กรอง spam: FP (อีเมลจริงหาย) แพงกว่า

อ่านตารางให้ถูกด้าน

```python
from sklearn.metrics import confusion_matrix

y_hat = (p_hat > 0.5).astype(int)   # labels from probabilities
cm = confusion_matrix(y, y_hat)     # rows = actual, columns = predicted
print(cm)
# [[9625   42]
#  [ 233  100]]
tn, fp, fn, tp = cm.ravel()         # always in this order
```

แต่ละตำราวางตารางไม่เหมือนกัน

- scikit-learn: แถว = ค่าจริง คอลัมน์ = ค่าที่ทาย เรียง 0 ก่อน 1
- ISLR Table 4.4: **แถว = ค่าที่ทาย** คอลัมน์ = ค่าจริง
- บางตำราและบางสไลด์วาง 1 ก่อน 0 ด้วย

อ่านหัวแถวและหัวคอลัมน์ทุกครั้งก่อนคำนวณ ในวิชานี้ใช้แบบ scikit-learn ตลอด

Metrics

ตัวเลขเดียวไม่พอ

ตัวชี้วัดจาก confusion matrix

ชื่อ	สูตร	ตอบคำถาม	`Default`, 0.5
accuracy	$(TP + TN)/n$	ทายถูกกี่ส่วนจากทั้งหมด	0.9725
recall (sensitivity, TPR)	$TP/(TP + FN)$	ในคนที่ผิดนัดจริง จับได้กี่ส่วน	0.3003
specificity (TNR)	$TN/(TN + FP)$	ในคนที่ไม่ผิดนัด ทายถูกกี่ส่วน	0.9957
precision	$TP/(TP + FP)$	ในคนที่ทายว่าผิดนัด ถูกกี่ส่วน	0.7042
FPR	$FP/(TN + FP)$	$= 1 -$ specificity	0.0043

recall กับ specificity ใช้ **แถว** (ค่าจริง) เป็นตัวหาร precision ใช้ **คอลัมน์** (ค่าที่ทาย) เป็นตัวหาร

ISLR §4.4.3 ลด threshold ลงเป็น 0.2 เราทำแบบเดียวกันกับ model ของเรา

	$\hat{y} = 0$	$\hat{y} = 1$
$y = 0$	TN $= 9404$	FP $= 263$
$y = 1$	FN $= 134$	TP $= 199$

1. คำนวณ recall, precision และ accuracy
2. เทียบกับที่ 0.5: ตัวไหนขึ้น ตัวไหนลง
3. ถ้าคุณเป็นธนาคาร จะเลือก threshold ไหน

	$t = 0.5$	$t = 0.2$
recall	0.3003	0.5976
precision	0.7042	0.4307
specificity	0.9957	0.9728
accuracy	0.9725	0.9603

- จับผู้ผิดนัดได้เพิ่ม 99 คน แลกกับ false alarm เพิ่ม 221 คน
- accuracy **ลดลง** แต่ถ้าการพลาดผู้ผิดนัดหนึ่งคนแพงกว่าการปฏิเสธลูกค้าดีหนึ่งคนมาก $t = 0.2$ อาจดีกว่า
- accuracy ให้น้ำหนักความผิดทั้งสองแบบเท่ากัน ซึ่งงานจริงแทบไม่เคยเป็นอย่างนั้น

F1 และการเลือกตัวชี้วัด

$$F_1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

harmonic mean: ถ้าตัวใดตัวหนึ่งต่ำ F_1 จะต่ำตาม บน `Default`: 0.4211 ที่ $t = 0.5$ และ 0.5006 ที่ $t = 0.2$

งาน	ตัวชี้วัดหลัก
คัดกรองโรค ตรวจจับการโกง	recall (อย่าพลาดคนที่เป็น)
กรอง spam แนะนำสินค้า	precision (สิ่งที่ทายว่าใช่ ต้องใช่จริง)
ต้องการสมดุลระหว่างสองอย่าง	F_1
กลุ่มใกล้เคียงกันและความผิดสองแบบแพงพอกัน	accuracy

ทาย “ไม่ผิดนัด” ทุกคน: accuracy 0.9667 แต่ recall $= 0$ ตัวชี้วัดที่ถูกต้องจะจับ model แบบนี้ได้ทันที

Train/test split

วัดบนข้อมูลที่ model ไม่เคยเห็น

ทำไม training set จึงหลอกเรา

- maximum likelihood เลือก $\hat{\beta}$ ให้เข้ากับ **ข้อมูลชุดนี้** มากที่สุด
- วัดบนข้อมูลชุดเดิมจึงได้ตัวเลขที่ดีเกินจริง ยิ่ง model ซับซ้อนยิ่งเกินมาก
- คาบที่ 7–9 เราเก็บ 12 เดือนสุดท้ายไว้ทดสอบ ข้อมูลที่ไม่มีลำดับเวลาให้ **สุ่ม** แบ่งแทน

กติกา

- ทุกอย่างที่ “เรียนจากข้อมูล” ต้องเรียนจาก train เท่านั้น: $\hat{\beta}$, ค่าเฉลี่ยและ SD ที่ใช้ standardize, threshold, การเลือกตัวแปร
- test set ใช้ **ครั้งเดียว** ตอนรายงานผล
- `stratify=y`: ให้สัดส่วนผู้ป่วยใน train และ test เท่ากัน

ใน Python

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=0)

scaler = StandardScaler().fit(X_tr)          # mean and sd from train only
model = LogisticRegression(C=np.inf, tol=1e-8)
model.fit(scaler.transform(X_tr), y_tr)
p_te = model.predict_proba(scaler.transform(X_te))[:, 1]
```

- `random_state=0`: แบ่งเหมือนเดิมทุกครั้งที่รัน ทุกคนในห้องได้ตัวเลขเดียวกัน
- `tol=1e-8`: ให้ค่าตรงกับ `statsmodels` ค่าเริ่มต้น `1e-4` หยุดเร็วไป $\hat{p}$ ต่างกันได้ถึง 0.0016

Framingham หลังแบ่ง

	จำนวน	เป็นโรค	สัดส่วน
ทั้งหมด (แถวที่ครบ)	3658	557	0.1523
train (75%)	2743	418	0.1524
test (25%)	915	139	0.1519

- `stratify=y` ทำให้สัดส่วนผู้ป่วยแทบเท่ากันทั้งสองส่วน
- บน test set การทาย “ไม่เป็นโรค” ทุกคนได้ accuracy 0.8481 นี่คือเส้นที่ต้องชนะ
- model ใช้ predictor ครบ 15 ตัวเหมือนคาบที่ 10

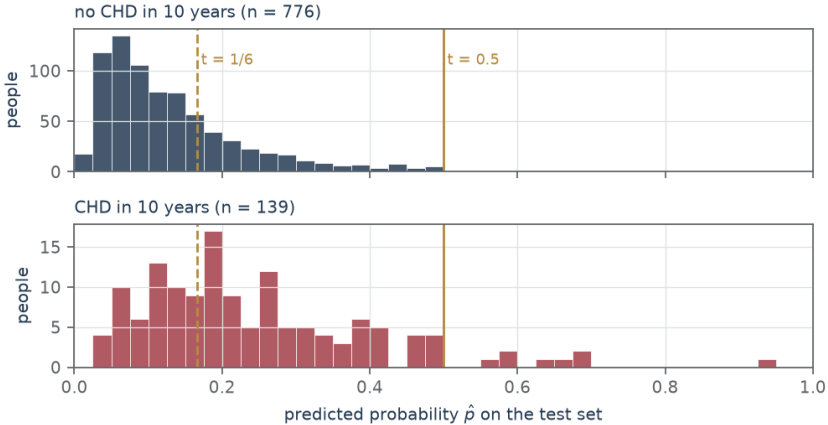
ผลบน test set ที่ $t = 0.5$

	$\hat{y} = 0$	$\hat{y} = 1$	รวม
$y = 0$	TN $= 774$	FP $= 2$	776
$y = 1$	FN $= 131$	TP $= 8$	139

	train	test
accuracy	0.8542	0.8546
recall	0.0766	0.0576
precision	0.6957	0.8000

ชนะการทาย “ไม่เป็นโรค” ทุกคน (0.8481) ไม่ถึง 1% และจับผู้ป่วยได้ 8 **จาก** 139 **คน** train กับ test ใกล้กัน ปัญหาจึงไม่ใช่ overfitting แต่เป็นที่ threshold

ทำไม 0.5 จึงจับได้น้อย



ผู้ป่วยมี $\hat{p}$ สูงกว่าจริง (มัธยฐาน 0.2005 เทียบกับ 0.1031) model **เรียงลำดับ** ได้ แต่เมื่อโรคพบเพียง 15% แทบไม่มีใครมี $\hat{p} > 0.5$

Threshold

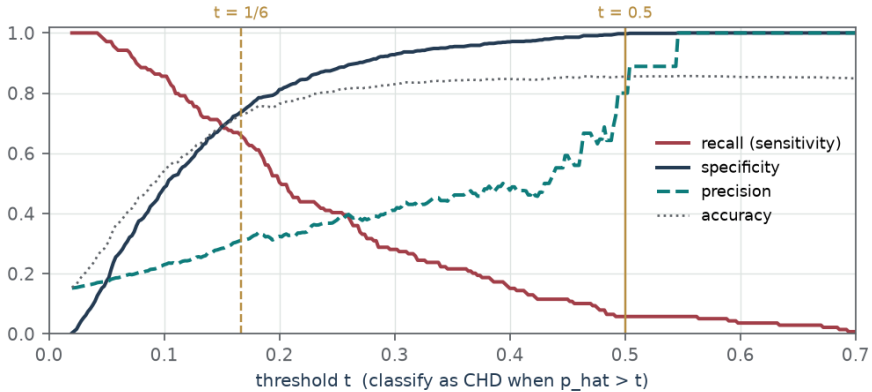
0.5 ไม่ใช่กฎธรรมชาติ

เลื่อน threshold บน test set

t	ทายว่าเป็นโรค	TP	FP	recall	precision	accuracy
0.5	10	8	2	0.0576	0.8000	0.8546
0.3	95	39	56	0.2806	0.4105	0.8295
0.2	216	70	146	0.5036	0.3241	0.7650
0.1	518	119	399	0.8561	0.2297	0.5421

- t ลดลง: ทายว่าเป็นโรคมากขึ้น **recall ขึ้น** precision และ accuracy ลง
- ไม่มี t ไหนดีที่สุดในทุกด้าน ต้องเลือกจาก **ต้นทุนของความผิดแต่ละแบบ**

Trade-off ทั้งเส้น



Framingham test set recall กับ specificity เคลื่อนสวนทางกันเสมอ precision ขึ้นตาม t แต่ไม่เรียบ เพราะเมื่อ t สูง ผู้ที่ถูกทายว่าเป็นโรคเหลือไม่กี่คน

เลือก threshold จากต้นทุน

ให้ c_{FP} = ต้นทุนของ false alarm หนึ่งครั้ง และ c_{FN} = ต้นทุนของการพลาดผู้ป่วยหนึ่งคน

ต้นทุนที่คาดหมาย ของคนที่มี $\hat{p}$

- ทายว่า 0: $\hat{p} \cdot c_{FN}$
- ทายว่า 1: $(1 - \hat{p}) \cdot c_{FP}$

ทายว่า 1 เมื่อ

$$\hat{p}\, c_{FN} > (1-\hat{p})\, c_{FP} \iff \hat{p} > \frac{c_{FP}}{c_{FP} + c_{FN}}$$

- ต้นทุนเท่ากัน: $t = 1/2$ คือค่าเริ่มต้นของ `predict`
- พลาดผู้ป่วยหนึ่งคน แย่เท่ากับ false alarm 5 ครั้ง:

$$t = \frac{1}{1+5} = \frac{1}{6} \approx 0.1667$$

สูตรนี้ใช้ได้เมื่อ $\hat{p}$ เป็นความน่าจะเป็นที่เชื่อถือได้ และ t ถูกกำหนด **ก่อน** ดู test set

ผลที่ $t = 1/6$ บน test set

	$\hat{y} = 0$	$\hat{y} = 1$
$y = 0$	TN $= 571$	FP $= 205$
$y = 1$	FN $= 47$	TP $= 92$

	$t = 0.5$	$t = 1/6$
recall	0.0576	0.6619
precision	0.8000	0.3098
accuracy	0.8546	0.7246
ต้นทุนรวม ($c_{FP} = 1, c_{FN} = 5$)	657	440

- จับผู้ป่วยได้ 92 จาก 139 คน แลกกับ false alarm 205 ครั้ง

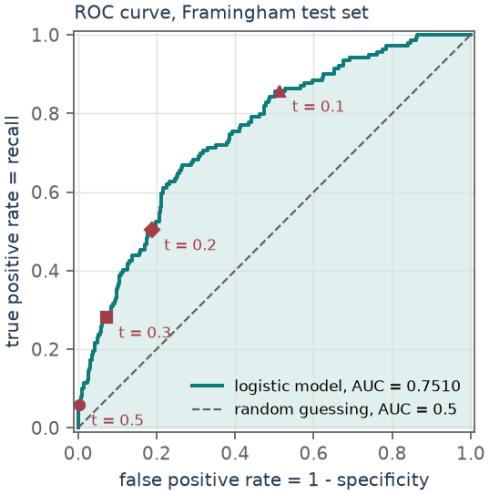
อย่าเลือก t จาก test set

ถ้าลองหลายค่าแล้วเลือกค่าที่ทำให้ test ดีที่สุด ตัวเลข test จะดีเกินจริงแบบเดียวกับ training set

ROC curve และ AUC

ดูทุก threshold พร้อมกัน

ROC curve



- แต่ละ t ให้จุดหนึ่งจุด: (FPR, recall)
- ไล่ t จาก 1 ลงไป 0 ได้เส้นจาก $(0, 0)$ ไป $(1, 1)$
- จุดสีแดงคือ 4 แถวในตารางเมื่อสักครู่
- เส้นประ: ทายแบบสุ่ม
- ยิ่งเส้นโค้งไปทางมุมซ้ายบน model ยิ่ง **แยกสองกลุ่มได้ดี**

ROC = receiver operating characteristic ชื่อมาจากงานเรดาร์

AUC: พื้นที่ใต้ ROC curve

- 0.5 = เดาสุ่ม, 1 = แยกสองกลุ่มได้สมบูรณ์
- Framingham test set: 0.7510 `Default` (training data): 0.9480

ความหมายที่จำง่าย

สุ่มผู้ป่วยมา 1 คน และคนที่ไม่ป่วยมา 1 คน AUC คือความน่าจะเป็นที่ model ให้ $\hat{p}$ ของ **ผู้ป่วยสูงกว่า**

- ตรวจได้ด้วยการนับ: ผู้ป่วย 139 คน $\times$ ไม่ป่วย 776 คน $= 107864$ คู่ เรียงถูก 0.7510 ของทุกคู่
- AUC ไม่ขึ้นกับ threshold จึงใช้ **เทียบ model** ได้ แต่ไม่ได้บอกว่าควรตัดที่ t เท่าใด

ใน Python

```python
from sklearn.metrics import roc_curve, roc_auc_score

fpr, tpr, thr = roc_curve(y_te, p_te)    # one point per threshold
print(f"{roc_auc_score(y_te, p_te):.4f}")             # 0.7510

# AUC = share of (case, non-case) pairs ranked correctly
pos = p_te[y_te.to_numpy() == 1]
neg = p_te[y_te.to_numpy() == 0]
print(f"{(pos[:, None] > neg[None, :]).mean():.4f}")  # 0.7510
```

ทั้งสองฟังก์ชันรับ **คะแนน** คือ $\hat{p}$ ของกลุ่ม 1 ไม่ใช่ป้ายชื่อ $0/1$

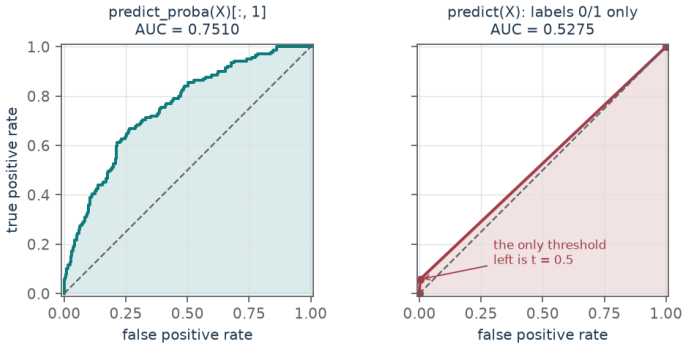
ลองทายก่อน

นักศึกษาคนหนึ่งเขียน

```
roc_auc_score(y_te, model.predict(scaler.transform(X_te)))
```

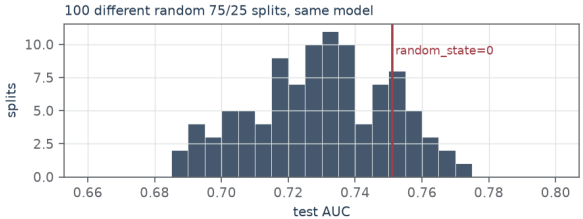
1. code นี้ error หรือไม่
2. ได้ AUC มากกว่า เท่ากับ หรือน้อยกว่า 0.7510
3. ถ้าได้ค่าที่ต่างไป ค่านั้นมาจากไหน

เฉลย: ไม่ error แต่ผิด



- `predict` คืน $0/1$ เหลือ threshold เดียวคือ 0.5 ROC จึงเหลือสองเส้นตรง และ AUC $= (\text{recall} + \text{specificity})/2 = 0.5275$
- `predict_proba(X)[:, 0]` คือ $\hat{p}$ ของกลุ่ม 0 ลำดับกลับด้าน ได้ $0.2490 = 1 - 0.7510$

ผลที่ต้องรายงานอย่างซื่อสัตย์



- model เดิม แบ่งข้อมูล 100 แบบ: test AUC ตั้งแต่ 0.6853 ถึง 0.7723 มัธยฐาน 0.7303 SD 0.0202
- `random_state=0` ให้ 0.7510 ซึ่ง **ค่อนข้างโชคดี** test set มีผู้ป่วยเพียง 139 คน
- ที่ split นี้ test AUC สูงกว่า train (0.7348) เป็นเรื่องบังเอิญ ไม่ใช่รูปแบบ

กำหนด split ก่อนดูผล และรายงานว่าตัวเลขจาก split เดียวแกว่งได้ประมาณ ± 0.02 การเฉลี่ยหลาย split คือ cross-validation (ISLR บทที่ 5) ซึ่งอยู่นอกขอบเขตวิชานี้

Multinomial logistic regression

เมื่อ Y มีสามกลุ่มขึ้นไป

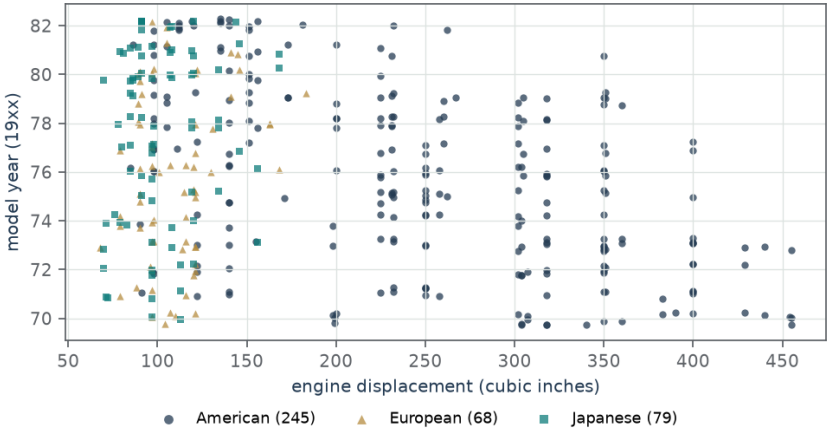
รถคันนี้ผลิตที่ไหน

ข้อมูล `Auto` (ISLR) 392 คัน ชุดเดียวกับปฏิบัติการที่ 5 ซึ่งเรา **ตัด `origin` ทิ้ง** เพราะเป็นหมวดหมู่ที่เก็บเป็นเลข $1/2/3$

`origin`	จำนวน	สัดส่วน	`displacement` เฉลี่ย	`year` เฉลี่ย
1 American	245	0.6250	247.5	75.59
2 European	68	0.1735	109.6	75.68
3 Japanese	79	0.2015	102.7	77.44

- `displacement` = ขนาดเครื่องยนต์ (ลูกบาศก์นิ้ว) `year` = ปีรุ่น 70–82 (ค.ศ. 1970–1982)
- เส้นที่ต้องชนะ: ทาย American ทุกคัน ได้ accuracy 0.6250

ดูข้อมูลก่อน



ไม่มีรถยุโรปหรือญี่ปุ่นคันใดเครื่องใหญ่กว่า 183 ลูกบาศก์นิ้ว แต่มีรถอเมริกัน 36 คันที่เครื่องไม่เกิน 122 รถยุโรปกับญี่ปุ่นปนกันเกือบทั้งหมด

1. OLS บนรหัส $1/2/3$

- บังคับลำดับ: European อยู่ “ระหว่าง” American กับ Japanese
- ค่าที่ fit ได้บน test: 0.63 ถึง 2.21
- ปัดเป็นจำนวนเต็มแล้ว **ไม่เคยทาย Japanese เลย**
- accuracy 0.5714 แพ้การทาย American ทุกคัน (0.6224)

2. แยก fit binary สามตัว

- American หรือไม่, European หรือไม่, Japanese หรือไม่
- ความน่าจะเป็นสามค่า **ไม่รวมกันเป็น 1**: ตั้งแต่ 0.8755 ถึง 1.2978
- 114 จาก 392 คันคลาดจาก 1 เกิน 0.05

ตัวอย่าง: `fiat 128` (รถยุโรป ปี 1973 เครื่อง 68) ได้ American 0.0494 European 0.6242 Japanese 0.6242 รวม 1.2978

Baseline-category logit

เลือกกลุ่มอ้างอิง (baseline) หนึ่งกลุ่ม ที่นี่คือ American ($k = 1$) แล้วเขียน log-odds ของแต่ละกลุ่ม **เทียบกับ baseline**

$$\log \frac{\Pr(Y = 2 \mid x)}{\Pr(Y = 1 \mid x)} = \beta_{02} + \beta_{12} x_1 + \beta_{22} x_2 \qquad \text{(European vs American)}$$

$$\log \frac{\Pr(Y = 3 \mid x)}{\Pr(Y = 1 \mid x)} = \beta_{03} + \beta_{13} x_1 + \beta_{23} x_2 \qquad \text{(Japanese vs American)}$$

- K กลุ่ม ใช้ $K - 1$ สมการ แต่ละสมการมีสัมประสิทธิ์ชุดของตัวเอง
- $K = 2$: เหลือสมการเดียว คือ logistic regression ของคาบที่ 10 พอดี
- ทุกสมการ fit **พร้อมกัน** ด้วย maximum likelihood ไม่ใช่แยก fit

จาก log-odds กลับเป็นความน่าจะเป็น: softmax

ให้ $\eta_k = \beta_{0k} + \beta_{1k}x_1 + \beta_{2k}x_2$ สำหรับ $k = 2, 3$ และ $\eta_1 = 0$ สำหรับ baseline

$$\Pr(Y = k \mid x) = \frac{e^{\eta_k}}{1 + e^{\eta_2} + e^{\eta_3}} \qquad \Pr(Y = 1 \mid x) = \frac{1}{1 + e^{\eta_2} + e^{\eta_3}}$$

- ทุกค่าอยู่ใน $(0, 1)$ และ **รวมกันเป็น 1 เสมอ**
- $K = 2$: $\Pr(Y = 2 \mid x) = e^{\eta}/(1 + e^{\eta}) = \sigma(\eta)$ คือ sigmoid
- likelihood: คูณ $\Pr(Y = y_i \mid x_i)$ ของกลุ่มที่เกิดขึ้นจริงของแต่ละคน หลักเดียวกับคาบที่ 10

ทาย $\hat{y}$ = กลุ่มที่มีความน่าจะเป็นสูงสุด (ไม่มี threshold เดียวแบบสองกลุ่ม)

ใน statsmodels

```
import pandas as pd
import statsmodels.api as sm

auto = pd.read_csv(AUTO_URL)                    # the pinned Lab 05 URL
auto["disp100"] = auto["displacement"] / 100    # units of 100 cubic inches

X = sm.add_constant(auto[["disp100", "year"]]) # same trap as Lec 10
fit = sm.MNLogit(auto["origin"], X).fit()
fit.params        # one column per non-baseline class
fit.predict(X)    # n x 3 probabilities; each row sums to 1
```

- baseline คือกลุ่มที่มีรหัสน้อยที่สุด (1 = American)
- คอลัมน์ `0` ของ `params` คือ European vs American คอลัมน์ 1 คือ Japanese vs American

ผลลัพธ์

	European vs American			Japanese vs American		
	coef	SE	p	coef	SE	p
`const`	14.3803	3.8840	0.0002	5.4989	3.9287	0.1616
`disp100`	-3.6485	0.5512	< 0.0001	-4.9855	0.7131	< 0.0001
`year`	-0.1331	0.0493	0.0069	0.0033	0.0501	0.9479

- log-likelihood -219.57 เทียบกับ -360.81 ของ model ที่มีแค่ intercept
- อ่าน SE, z และ p เหมือนคาบที่ 10 ทุกอย่าง แค่มีสองคอลัมน์

- เครื่องใหญ่ขึ้น 100 ลูกบาศก์นิ้ว (ปีรุ่นเท่าเดิม):
  - odds ของ European **เทียบกับ American** คูณ $e^{-3.6485} = 0.0260$
  - odds ของ Japanese เทียบกับ American คูณ $e^{-4.9855} = 0.0068$
  - Japanese เทียบกับ European: $e^{-4.9855-(-3.6485)} = 0.2626$ ไม่ต้อง fit ใหม่
- year ในสมการ Japanese: $p = 0.9479$ เมื่อขนาดเครื่องเท่ากัน ไม่มีหลักฐานว่าปีรุ่นแยก Japanese จาก American ได้

เครื่องหมายของ β ไม่ได้บอกทิศของ $\Pr(Y = k)$

β ของ European ติดลบ แต่ปี 1972 เมื่อเครื่องโตจาก 70 เป็น 100 ลูกบาศก์นิ้ว $\Pr(\text{European})$ **เพิ่ม** จาก 0.4737 เป็น 0.5023 เพราะ Japanese ลดเร็วกว่า ($-4.9855 < -3.6485$) β บอกแค่ odds เทียบกับ baseline

รถเครื่อง 100 ลูกบาศก์นิ้ว (`disp100` $= 1$) ปีรุ่น 1980 (`year` $= 80$)

ใช้สัมประสิทธิ์ในตารางเมื่อสักครู่ คำนวณความน่าจะเป็นของทั้งสามกลุ่ม แล้วบอกว่า model ทายว่ามาจากไหน

1. $\eta_E = \beta_{0,E} + \beta_{1,E} \cdot 1 + \beta_{2,E} \cdot 80$ และ η_J แบบเดียวกัน
2. e^{η_E}, e^{η_J} และตัวหาร $1 + e^{\eta_E} + e^{\eta_J}$
3. หารทีละกลุ่ม แล้วตรวจว่ารวมกันได้ 1

$$\eta_E = 14.3803 + (-3.6485)(1) + (-0.1331)(80) = 0.0838 \qquad e^{\eta_E} = 1.0874$$
$$\eta_J = 5.4989 + (-4.9855)(1) + (0.0033)(80) = 0.7774 \qquad e^{\eta_J} = 2.1758$$

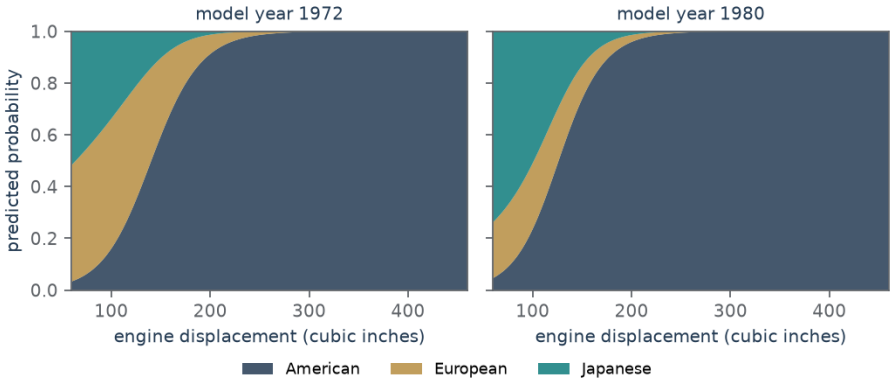
$$\text{ตัวหาร} = 1 + 1.0874 + 2.1758 = 4.2632$$

	American	European	Japanese
ด้วยมือ	0.2346	0.2551	0.5104
`fit.predict()`	0.2350	0.2547	0.5103

- model ทาย **Japanese** แต่มั่นใจเพียง 0.51
- ค่าต่างกันเล็กน้อยเพราะสัมประสิทธิ์ปัดเหลือ 4 ตำแหน่ง แล้วคูณ 80

รถเครื่อง 200 ปีรุ่น 1976: American 0.9408 (`predict()` ได้ 0.9410)

ความน่าจะเป็นทั้งสามกลุ่ม



ความสูงของแต่ละแถบคือความน่าจะเป็น รวมกันเป็น 1 ทุกจุด เครื่องเกิน 200 เกือบแน่นอนว่าเป็นรถอเมริกัน เครื่องเล็ก ปีรุ่นเป็นตัวบอกว่า European หรือ Japanese มีโอกาสมากกว่ากัน

ใน scikit-learn

```python
from sklearn.linear_model import LogisticRegression

X2 = auto[["disp100", "year"]]
sk = LogisticRegression(C=np.inf, tol=1e-8, max_iter=10_000)
sk.fit(X2, auto["origin"])      # 3 classes -> softmax automatically

sk.coef_.shape                  # (3, 2): one row per class, no baseline
sk.coef_ - sk.coef_[0]          # minus American's row -> MNLogit's slopes
sk.predict_proba(X2)            # same probabilities as fit.predict(X)
```

แถวของ coef_	intercept_	disp100	year
American	-6.6264	2.8780	0.0433
European	7.7539	-0.7705	-0.0899
Japanese	-1.1275	-2.1075	0.0466

แต่ละคอลัมน์รวมกันได้ 0 ลบแถว American แล้วได้ 14.3803, -3.6485, -0.1331 (European) และ 5.4989, -4.9855, 0.0033 (Japanese) ตรงกับ `statsmodels`

Confusion matrix สามกลุ่ม

fit ใหม่บน 75% (294 คัน, `stratify`) แล้วทายอีก 98 คัน

	ทาย American	ทาย European	ทาย Japanese	รวม	recall
American	**53**	3	5	61	0.8689
European	6	**6**	5	17	0.3529
Japanese	5	3	**12**	20	0.6000
รวม	64	12	22	98	
precision	0.8281	0.5000	0.5455		

- accuracy 0.7245 เทียบกับ 0.6224 ของการทาย American ทุกคัน
- recall ของแต่ละกลุ่ม = แนวทแยง ÷ ผลรวมแถว precision = แนวทแยง ÷ ผลรวมคอลัมน์ นิยามเดิม แค่มองทีละกลุ่ม

ผลที่ต้องรายงานอย่างซื่อสัตย์

สิ่งที่ model ทำได้

- แยกรถอเมริกันออกจากรถนำเข้าด้วยขนาดเครื่อง (recall 0.8689)
- ความน่าจะเป็นรวมกันเป็น 1 ตีความได้

สิ่งที่ model ทำไม่ได้

- รถยุโรปทายถูก 6 จาก 17 คัน อีก 6 คันถูกทายว่าอเมริกัน 5 คันว่าญี่ปุ่น
- ขนาดเครื่องกับปีรุ่นไม่มีข้อมูลที่แยกยุโรปจากญี่ปุ่น

เพิ่ม predictor ได้ แต่ `weight`, `mpg` และ `displacement` สัมพันธ์กันสูง (`displacement` กับ `weight`: 0.933)

ระวัง collinearity แบบคาบที่ 6 test set มีรถยุโรปเพียง 17 คัน ตัวเลขรายกลุ่มจึงแกว่งได้มาก

สรุปคาบที่ 11

1. confusion matrix แยกความผิดเป็น FP และ FN ซึ่งมีต้นทุนต่างกัน (scikit-learn: แถว = ค่าจริง)
2. recall = TP/(TP+FN), precision = TP/(TP+FP), specificity = TN/(TN+FP), F_1 = harmonic mean ของ precision กับ recall
3. accuracy หลอกได้เมื่อกลุ่มไม่สมดุล: ทาย “ไม่ป่วย” ทุกคนก็ได้ 85%
4. วัดบน test set ที่แบ่งไว้ก่อน (`stratify=y`) และเรียนทุกอย่างจาก train เท่านั้น รวมถึง scaler
5. ลด threshold: recall ขึ้น precision ลง เลือก $t = c_{FP}/(c_{FP} + c_{FN})$ จากต้นทุน ไม่ใช่จาก test set
6. ROC = (FPR, recall) ทุก threshold, AUC = ความน่าจะเป็นที่ผู้ป่วยได้ $\hat{p}$ สูงกว่าคนไม่ป่วย
7. AUC ต้องคำนวณจาก `predict_proba(X)[:, 1]` ไม่ใช่ `predict(X)` (0.5275 แทน 0.7510)
8. ตัวเลขจาก split เดียวแกว่งได้ (AUC 0.6853–0.7723 บน 100 split)
9. multinomial: $K - 1$ สมการ log-odds เทียบ baseline แล้วแปลงด้วย softmax ความน่าจะเป็นรวมเป็น 1
10. e^{β} คือตัวคูณ odds **เทียบกับ baseline** ไม่ใช่ทิศทางของ $\Pr(Y = k)$

สิ่งที่ควรทำได้ด้วยมือก่อนสอบปลายภาค

- จาก confusion matrix 2×2: คำนวณ accuracy, recall, specificity, precision และ F_1
- บอกทิศทางของแต่ละตัวชี้วัดเมื่อเลื่อน threshold
- หา threshold จากต้นทุน c_{FP} และ c_{FN}
- อ่าน ROC curve: จุดหนึ่งจุดคือ threshold หนึ่งค่า และอธิบายความหมายของ AUC
- จากสัมประสิทธิ์ multinomial: คำนวณ η_k, softmax และกลุ่มที่ model ทาย
- ตีความ e^{β} เทียบกับ baseline และหา odds ratio ระหว่างสองกลุ่มที่ไม่ใช่ baseline
- จาก confusion matrix 3×3: recall และ precision รายกลุ่ม

สอบปลายภาคครอบคลุมคาบที่ 7–11

อ่านเพิ่มเติม

James, Witten, Hastie & Tibshirani, An Introduction to Statistical Learning

- §4.3.5 logistic regression สำหรับมากกว่าสองกลุ่ม
- §4.4.3 ตั้งแต่ Table 4.4: confusion matrix, threshold 0.2, ROC และ Table 4.6–4.7 ที่รวมชื่อตัวชี้วัด (ตัวอย่างในหนังสือใช้ LDA แต่ข้ามสูตรของ LDA ได้ แนวคิดการประเมินเหมือนกัน)
- ฉบับพิมพ์ครั้งที่ 2 §4.3.5 อธิบาย multinomial logistic regression และ softmax ละเอียดกว่า

ปฏิบัติการที่ 11 ทำทุกขั้นบน Framingham และ `Auto` สัปดาห์หน้า: ทบทวนคาบที่ 7–11 และ mock final